사내 기술문서 질의응답 에이전트
KnowBridge 프로젝트 문서 (텍스트판)

LangGraph 기반 Agentic RAG 아키텍처와 Contextual Retriever를 통한 청크 문맥 유실 문제 해결

문서 번호: KB-PRJ-001-T

문서 버전: v1.2 (최종)

작성일: 2026-11-09

작성 조직: AI플랫폼팀 (가상 조직)

프로젝트 기간: 2026-07-01 ~ 2026-11-13 (가상 일정)

핵심 기술: LangGraph, RAG, Contextual Retrieval, 하이브리드 검색(Dense + BM25), 리랭킹

예시 문서 안내. 이 문서는 벡터 DB 적재와 RAG 검색 테스트를 위해 만든 예시 프로젝트 문서다. 프로젝트명, 조직, 일정, 내부 설문 결과, 평가 수치는 모두 가상이다. 단, 7장에 인용한 Anthropic의 Contextual Retrieval 실험 수치와 비용 추정치는 공개 자료이며, 출처를 본문과 참고문헌에 표시했다.

텍스트판 안내. 이 문서는 KnowBridge 프로젝트 문서(KB-PRJ-001)의 텍스트 전용판이다. 원본의 그림과 표를 모두 문장과 목록으로 바꿔 적었다. 이미지나 표 구조를 읽지 못하는 파서에서도 내용이 그대로 추출되도록 하기 위해서다.

문서 정보 및 목차

문서 정보

문서명은 “KnowBridge 프로젝트 문서 — LangGraph 기반 Agentic RAG와 Contextual Retriever(텍스트판)”이다.

대상 독자는 KnowBridge를 개발·운영하는 개발자, 운영자, 프로젝트 관리자다.

이 문서의 목적은 KnowBridge의 아키텍처, 설계 결정, 문제 해결 과정, 평가 결과를 기록하는 것이다.

보안 등급은 사내 공개(예시 문서)다.

개정 이력

- v0.1 (2026-07-03): 초안 작성. 프로젝트 배경과 요구사항을 정리했다.
- v0.5 (2026-08-21): RAG 프로토타입의 청크 문맥 유실 문제 분석 결과를 추가했다.
- v1.0 (2026-09-30): Contextual Retriever 설계와 적용 결과를 반영했다.
- v1.2 (2026-11-09): LangGraph 재검색 루프, 평가 결과, 운영 절차를 보완했다.

목차

1. 프로젝트 개요
2. 요구사항 및 추진 일정
3. 시스템 아키텍처
4. 기술 스택
5. 문서 수집·색인 파이프라인
6. 문제 정의: 청크 분할 시 문맥 유실
7. Contextual Retriever 설계와 적용
8. LangGraph 에이전트 워크플로우
9. 성능 평가
10. 운영 및 리스크 관리
11. 결론, 향후 계획, 참고문헌

문서 작성 규칙 (벡터 DB 적재 고려)

이 문서는 청크 단위로 잘려 벡터 DB에 저장되는 것을 전제로 작성했다.

각 장의 첫 문단에는 프로젝트명(KnowBridge)과 해당 장의 주제를 다시 적는다. 그래서 한 장만 따로 검색되어도 무엇에 관한 내용인지 알 수 있다.

원본 문서의 그림과 표는 이 텍스트판에서 문장과 목록으로 풀어 썼다.

1. 프로젝트 개요

1.1 배경

KnowBridge는 사내 기술문서를 대상으로 자연어 질문에 답하는 질의응답 에이전트다.

사내 기술문서는 Confluence 위키, Git 저장소의 README, PDF 운영 매뉴얼에 흩어져 있다.

개발자는 장애 대응 중에 설정값이나 운영 절차를 찾느라 많은 시간을 쓴다.

사내 설문(가상)에서 개발자는 문서 검색에 주당 평균 3.2시간을 쓴다고 답했다.

1.2 해결하려는 문제

첫째, 기존 키워드 검색은 질문과 문서의 표현이 다르면 문서를 찾지 못한다.

둘째, 7월에 만든 RAG 프로토타입은 문서를 잘게 쪼개면서 청크의 문맥이 사라졌다.

문맥이 사라진 청크는 검색되지 않거나, 다른 서비스의 내용과 섞여 오답을 만들었다.

KnowBridge는 두 번째 문제를 Contextual Retriever로 해결하는 것을 핵심 과제로 삼았다.

1.3 목표 지표 (KPI)

KnowBridge의 목표 지표는 4개다. 아래 수치는 모두 가상이다.

검색 실패율@20

정의: 정답 청크가 상위 20개 후보에 없는 질문의 비율이다.

프로토타입은 15.0%였다. 목표는 7% 이하였다. 최종 결과는 5.2%로 목표를 달성했다.

답변 정확도

정의: 평가자가 정답으로 판정한 답변의 비율이다.

프로토타입은 70.0%였다. 목표는 85% 이상이었다. 최종 결과는 88.5%로 목표를 달성했다.

근거 충실도

정의: 답변 내용이 검색 문서로 뒷받침되는 정도이며, 0에서 1 사이 값이다.

프로토타입은 0.80이었다. 목표는 0.90 이상이었다. 최종 결과는 0.93으로 목표를 달성했다.

P95 응답 시간

정의: 전체 요청 중 95번째 백분위의 응답 시간이다.

프로토타입은 1.8초였다. 목표는 4초 이하였다. 최종 결과는 3.4초로 목표를 달성했다.

최종 결과의 답변 정확도와 근거 충실도는 LangGraph 재검색 루프까지 포함한 전체 시스템 기준이다. 검색 실패율@20은 리랭킹까지 적용한 검색 단계 기준이다.

1.4 프로젝트 범위

범위에 포함한 항목은 4개다.

- Confluence, Git README, PDF 매뉴얼의 텍스트와 표를 색인한다.
- 웹 UI와 슬랙 봇으로 질의응답을 제공한다.
- 문서 권한(acl)에 따라 검색 결과를 필터링한다.

- 문서가 바뀌면 증분 색인한다.

범위에서 제외한 항목은 4개다.

- 이미지 안의 글자 인식(OCR)은 하지 않는다.
- 문서를 자동으로 작성하거나 수정하는 기능은 만들지 않는다.
- 외부 고객 대상 서비스는 제공하지 않는다.
- 초 단위 실시간 색인 반영은 하지 않는다.

2. 요구사항 및 추진 일정

이 장은 KnowBridge의 기능 요구사항, 비기능 요구사항, 추진 일정을 정리한다.

2.1 기능 요구사항

- FR-01 (필수): 사용자는 자연어로 질문하고 답변을 받는다.
- FR-02 (필수): 답변에는 근거 문서의 제목, 절, 링크를 함께 표시한다.
- FR-03 (필수): 사용자 권한으로 볼 수 없는 문서는 검색 결과에서 제외한다.
- FR-04 (필수): 근거 문서가 부족하면 추측하지 않고 “답변 불가”를 안내한다.
- FR-05 (높음): 문서가 바뀌면 바뀐 문서만 다시 색인한다.
- FR-06 (보통): 이전 대화 맥락을 반영해 후속 질문에 답한다.

2.2 비기능 요구사항

- NFR-01: P95 응답 시간은 4초 이하로 유지한다. Prometheus 지연 히스토그램으로 측정한다.
- NFR-02: 동시 사용자 200명을 처리한다. 부하 테스트로 검증한다.
- NFR-03: 로그에 남는 개인정보는 마스킹한다. 로그 샘플 점검으로 확인한다.
- NFR-04: 모든 요청은 노드 단위로 추적할 수 있어야 한다. LangSmith 트레이스로 확인한다.

2.3 추진 일정

프로젝트는 2026년 7월 1일에 시작해 11월 13일에 운영 배포를 마쳤다. 일정은 가상이다.

단계별 일정은 다음과 같다.

1. 요구사항 분석: 7월 1일 ~ 7월 14일 (2주)
2. RAG 프로토타입: 7월 15일 ~ 8월 7일 (약 3.5주)
3. 문맥 유실 원인 분석: 8월 10일 ~ 8월 21일 (2주)
4. Contextual Retriever 구현: 8월 24일 ~ 9월 18일 (약 4주)
5. LangGraph 에이전트 구현: 9월 14일 ~ 10월 9일 (약 4주)
6. 평가 및 튜닝: 10월 5일 ~ 10월 30일 (약 4주)
7. 운영 배포: 11월 2일 ~ 11월 13일 (2주)

Contextual Retriever 구현과 LangGraph 에이전트 구현은 9월 14일부터 18일까지 겹쳐 진행했다.

LangGraph 에이전트 구현과 평가 및 튜닝은 10월 5일부터 9일까지 겹쳐 진행했다.

3. 시스템 아키텍처

KnowBridge 아키텍처는 온라인 질의응답 경로와 오프라인 색인 경로로 나뉜다.

아래는 원본 문서의 아키텍처 그림을 계층별 텍스트로 옮긴 것이다. 위에서 아래 순서로 적었다.

3.1 계층 구성

계층 1. 사용자 채널

사용자는 웹 UI 또는 슬랙 봇으로 질문한다. 두 채널 모두 API 서버로 요청을 보낸다.

계층 2. API 서버

API 서버는 FastAPI로 만들었다. 사용자 인증, 권한 확인, 스트리밍 응답을 담당한다.

API 서버 옆에는 Redis 응답 캐시가 있다. API 서버는 같은 질문이 오면 Redis에서 캐시된 답변을 먼저 찾는다.

캐시에 답이 없으면 요청을 LangGraph 에이전트 오케스트레이터로 넘긴다.

계층 3. LangGraph 에이전트 오케스트레이터

오케스트레이터는 LangGraph의 StateGraph로 구현했다.

내부 처리 순서는 질의 분류 → 질의 재작성 → 하이브리드 검색 → 리랭킹 → 문서 평가 → 답변 생성 → 근거 검증이다.

문서 평가에서 관련 문서가 부족하면 질의 재작성 단계로 돌아간다. 이 재검색 루프는 최대 2회까지 반복한다.

오케스트레이터는 아래의 검색 계층과 LLM 계층을 호출한다.

계층 4-1. 검색 계층

검색 계층은 세 구성 요소로 이루어진다.

- Qdrant: Dense 벡터 검색을 담당한다.
- OpenSearch: Nori 분석기를 쓰는 BM25 키워드 검색을 담당한다.
- Reranker: bge-reranker-v2-m3 모델로 검색 결과를 다시 정렬한다.

계층 4-2. LLM 계층

LLM 계층은 Claude API다. 질의 재작성, 문서 평가, 답변 생성, 문맥 생성에 쓰인다.

계층 5. 오프라인 색인 파이프라인

색인 파이프라인은 배치와 증분 방식으로 실행된다.

처리 순서는 문서 소스(Confluence·Git·PDF) → 파싱·정제(표를 마크다운으로 변환) → 구조 기반 청킹 → Contextual Retriever → 임베딩(bge-m3) → 색인 적재다.

색인 적재 단계는 청크, 벡터, BM25 색인을 검색 계층의 Qdrant와 OpenSearch에 넣는다.

Contextual Retriever 단계는 LLM 계층의 Claude API를 호출한다.

보조 구성 요소

관측 도구는 LangSmith 트레이스, Prometheus, Grafana다.

메타데이터 저장소는 PostgreSQL이다. 문서 버전, 내용 해시, 권한 정보를 저장한다.

3.2 온라인 경로 (질의응답) 요약

사용자 질문 → API 서버(인증·권한 확인) → Redis 캐시 확인 → LangGraph 오케스트레이터 → 검색 계층과 LLM 계층 호출 → 출처가 붙은 답변을 스트리밍으로 반환.

3.3 오프라인 경로 (색인) 요약

문서 수집 → 파싱 → 청킹 → 청크별 문맥 생성 → 문맥을 붙인 청크 임베딩 → Qdrant와 OpenSearch에 적재.

4. 기술 스택

이 장은 KnowBridge에 사용한 기술과 선정 이유를 정리한다.

4.1 구성 요소별 기술과 선정 이유

- **에이전트 오케스트레이션 — LangGraph:** 조건 분기와 반복 루프를 그래프로 명시할 수 있다. 노드 간 상태를 하나의 State 객체로 관리한다.
- **LLM 연동 — LangChain:** 리트리버, 프롬프트 템플릿, 모델 호출을 같은 인터페이스로 다룰 수 있다.
- **LLM — Claude (Anthropic API):** 문맥 생성, 질의 재작성, 문서 평가, 답변 생성에 사용한다. 프롬프트 캐싱으로 문맥 생성 비용을 줄인다.
- **임베딩 모델 — BAAI/bge-m3:** 한국어를 포함한 다국어를 지원한다. 최대 8,192 토큰 입력을 받는다. Dense 벡터 차원은 1,024다.
- **벡터 DB — Qdrant:** 페이로드 필터로 문서 권한(acl) 조건을 검색과 함께 적용할 수 있다.
- **키워드 검색 — OpenSearch + Nori 분석기:** Nori는 한국어 형태소 분석기다. 형태소 단위 BM25 검색으로 고유명사와 설정 키를 정확히 찾는다.
- **리랭커 — BAAI/bge-reranker-v2-m3:** 질의와 문서를 함께 입력하는 Cross-Encoder 방식이다. 다국어를 지원한다.
- **API 서버 — FastAPI:** 비동기 처리와 스트리밍 응답을 지원한다.
- **캐시 — Redis:** 같은 질문의 반복 요청에 캐시된 답변을 돌려준다.
- **메타데이터 — PostgreSQL:** 문서 버전, 내용 해시, 권한 정보를 저장한다.
- **관측 — LangSmith, Prometheus, Grafana:** LangSmith로 노드별 입력·출력을 추적한다. Prometheus와 Grafana로 지연과 오류율을 본다.
- **평가 — Ragas와 사람 평가:** Ragas로 근거 충실도(faithfulness)를 측정한다. 답변 정확도는 사람이 판정한다.

4.2 설계 원칙

첫째, 검색 품질 문제는 생성 단계가 아니라 색인 단계에서 먼저 해결한다.

둘째, 키워드 검색과 의미 검색을 함께 쓴다. 설정 키나 오류 코드는 키워드 검색이 강하다. 표현이 다른 질문은 의미 검색이 강하다.

셋째, 근거가 부족하면 답하지 않는다. 오답보다 “답변 불가”가 운영상 안전하다.

넷째, 라이브러리 버전은 고정한다. LangGraph와 LangChain은 버전에 따라 API가 바뀔 수 있기 때문이다.

5. 문서 수집·색인 파이프라인

KnowBridge 색인 파이프라인은 6단계로 구성된다. 4단계가 Contextual Retriever다.

5.1 단계별 처리

1. **수집:** Confluence API, Git 저장소, PDF 매뉴얼에서 문서를 가져온다.
2. **파싱·정제:** 머리글과 바닥글을 제거한다. 표는 마크다운 표로 변환한다.
3. **청킹:** 헤딩 단위로 먼저 나누고, 400 토큰 길이와 50 토큰 중첩으로 다시 나눈다.
4. **문맥 생성:** Contextual Retriever가 청크마다 설명 문맥을 만든다.
5. **임베딩:** bge-m3로 1,024차원 Dense 벡터를 만든다.
6. **색인:** 벡터는 Qdrant에, 텍스트는 OpenSearch에 적재한다.

2단계부터 6단계까지의 결과는 메타데이터 저장소(PostgreSQL)에 기록한다. 기록 항목은 doc_id, version, section_path, acl, content_hash, updated_at이다.

5.2 청킹 전략

청킹은 문서 구조를 먼저 따르고, 그다음 토큰 길이로 자른다. 설정값과 이유는 다음과 같다.

- **1차 분할 기준 — 마크다운 헤딩(#, ##, ###):** 절 경계에서 자르면 한 청크에 한 주제가 담긴다.
- **청크 크기 — 최대 400 토큰:** 프로토타입 실험에서 검색 정밀도와 문맥량의 균형이 가장 좋았다(가상).
- **중첩 — 50 토큰:** 문장이 청크 경계에서 잘리는 문제를 줄인다.
- **표 처리 — 표 하나를 청크 하나로 유지:** 표를 자르면 헤더와 값의 대응이 깨진다.
- **코드 블록 — 분할 금지:** 설정 파일이나 명령어가 중간에 잘리지 않게 한다.

5.3 청크 메타데이터 예시

각 청크는 아래 메타데이터와 함께 저장된다. `acl`은 검색 시 권한 필터에 쓰인다.

```
{
  "chunk_id": "pgcore-v3.2-manual#4.3-002",
  "doc_id": "pgcore-v3.2-manual",
  "title": "PG-Core v3.2 운영 매뉴얼",
  "section_path": "4. 장애 대응 > 4.3 결제 재시도 정책",
  "version": "3.2",
  "acl": ["payments-dev", "sre"],
  "content_hash": "sha256:9f2c...",
  "context": "이 청크는 PG-Core v3.2 운영 매뉴얼 4.3절 결제 재시도 정책의 일부이며 …",
  "text": "이 값을 false로 설정하면 재시도가 비활성화된다. …",
  "updated_at": "2026-09-18T10:12:00+09:00"
}
```

`context` 필드는 Contextual Retriever가 생성한 문맥이다. 원본 `text`와 분리해 저장한다. 그래서 문맥만 다시 생성하거나 품질을 점검하기 쉽다.

6. 문제 정의: 청크 분할 시 문맥 유실

6.1 현상

KnowBridge 프로토타입은 문서를 400 토큰 청크로 나눠 임베딩했다.

그런데 원문에서 떨어진 청크는 무엇에 관한 내용인지 알 수 없는 경우가 많았다.

6.2 문맥 유실 예시

원문 문서

문서 제목은 “PG-Core v3.2 운영 매뉴얼”이다. 해당 절은 “4.3 결제 재시도 정책”이다.

본문은 다음과 같다. “결제 승인 요청이 타임아웃되면 PG-Core는 자동으로 재요청한다. 설정 키는 retry.enabled이다. 이 값을 false로 설정하면 재시도가 비활성화된다. 기본값은 true이며 최대 3회 재시도한다.”

분할 결과

청크 #1에는 문서 제목, 절 제목, “설정 키는 retry.enabled”라는 문장이 들어갔다.

청크 #2에는 “이 값을 false로 설정하면 재시도가 비활성화된다. 기본값은 true이며 최대 3회 재시도한다.”만 들어갔다.

검색 실패

사용자는 “PG-Core v3.2에서 결제 재시도를 끄는 방법”을 물었다. 정답은 청크 #2에 있다.

하지만 청크 #2에는 “PG-Core”, “결제”라는 단어가 없다. “이 값”이 retry.enabled라는 정보도 없다.

그래서 임베딩 유사도와 BM25 점수가 모두 낮았다. 청크 #2는 검색 순위 밖으로 밀려났다.

Contextual Retriever 적용 후

청크 #2 앞에 다음 문맥이 붙는다. “이 청크는 PG-Core v3.2 운영 매뉴얼 4.3절 ‘결제 재시도 정책’의 일부이며, 설정 키 retry.enabled로 결제 승인 타임아웃 시 자동 재시도를 끄는 방법을 설명한다.”

이제 청크에 서비스명, 버전, 주제어, 설정 키가 모두 들어 있다. 그래서 같은 질문으로 청크 #2가 검색된다.

6.3 원인 분석

프로토타입에서 검색에 실패한 질문 120건을 분석했다. 아래 비율은 가상 수치다.

- **지시어 의존 (38%):** “이 값”, “해당 서비스”, “위 설정”이 무엇을 가리키는지 청크 안에 없다.
- **주체 누락 (27%):** 서비스명이나 시스템명이 문서 제목에만 있고 청크에는 없다.
- **버전·시점 누락 (19%):** v3.1과 v3.2 문서의 청크 내용이 같아 구분되지 않는다.
- **상위 조건 누락 (16%):** “운영 환경에서는” 같은 조건이 상위 절에만 있다.

6.4 대안 검토

- **청크 크기 확대:** 구현이 쉽다. 하지만 여러 주제가 섞여 검색 정밀도가 떨어진다.
- **제목·절 경로 앞붙이기:** 비용이 거의 없다. 하지만 “이 값” 같은 지시어의 대상은 풀지 못한다.
- **문서 요약 앞붙이기:** 문서 단위 주제를 알려준다. 하지만 모든 청크에 같은 요약이 붙는다. Anthropic은 이 방식의 효과가 매우 제한적이었다고 보고했다 [1].
- **Contextual Retrieval (채택):** 청크마다 맞춤 문맥을 만든다. 색인할 때 LLM 호출 비용이 든다(7.3절).

7. Contextual Retriever 설계와 적용

7.1 개념

KnowBridge의 Contextual Retriever 모듈은 Anthropic이 2024년 9월에 공개한 Contextual Retrieval 기법을 구현한 것이다 [1].

핵심은 청크마다 문서 전체를 참고해 짧은 설명 문맥을 LLM으로 만들고, 그 문맥을 청크 앞에 붙이는 것이다.

Anthropic 자료에 따르면 이 문맥은 보통 50~100 토큰이다. 문맥을 붙인 청크로 임베딩(Contextual Embedding)과 BM25 색인(Contextual BM25)을 모두 만든다 [1].

7.2 처리 흐름

1. 문서 전체(예: 약 6천 토큰 분량의 PG-Core v3.2 운영 매뉴얼)와 청크 하나(예: 청크 #2)를 준비한다.
2. 두 입력을 문맥 생성 프롬프트에 넣어 Claude를 호출한다. 문서 부분에는 프롬프트 캐싱을 적용한다.
3. Claude는 50~100 토큰의 문맥을 만든다. 문맥에는 문서명, 절 제목, 서비스명, 청크의 역할이 들어간다.
4. 생성된 문맥과 원본 청크를 이어 붙여 contextualized_text를 만든다.
5. contextualized_text를 임베딩해 Qdrant에 넣는다. 이것이 Contextual Embedding이다.
6. 같은 contextualized_text를 OpenSearch에 넣는다. 이것이 Contextual BM25다.

7.3 문맥 생성 프롬프트

아래 프롬프트는 Anthropic 공개 자료의 구조를 참고해 한국어로 새로 작성했다.

```
<document>{문서 전체}</document>
다음은 위 문서에서 잘라낸 청크다.
<chunk>{청크 본문}</chunk>

이 청크가 문서 전체에서 어떤 위치와 의미를 갖는지 검색 품질 향상을 위해 짧게 설명하라.
문서명, 서비스명, 버전, 상위 절 제목을 포함하라. 문서에 없는 사실을 만들지 마라. 설명 문장만 출력하라.
```

7.4 비용 최적화: 프롬프트 캐싱

청크마다 문서 전체를 다시 입력하면 비용이 커진다.

그래서 같은 문서의 청크를 연속으로 처리하고, 문서 부분에 프롬프트 캐싱을 적용한다 [5].

Anthropic은 프롬프트 캐싱을 쓰면 문맥 생성 비용이 문서 100만 토큰당 약 1.02달러라고 추정했다 [1]. 이 값은 공개 당시의 가정과 가격 기준이다.

KnowBridge의 비용 추정은 다음과 같다. 문서 규모는 가상이다.

- 색인 대상 문서 수: 12,000건
- 문서당 평균 길이: 약 6,000 토큰
- 전체 문서 토큰: 약 7,200만 토큰
- Anthropic 공개 추정치(100만 토큰당 1.02달러)를 적용한 1회 비용: 약 73달러

실제 비용은 사용 모델, 현재 가격, 청크 크기, 문서 길이에 따라 달라진다.

7.5 하이브리드 검색, RRF, 리랭킹

KnowBridge는 문맥이 붙은 청크를 Dense 검색과 BM25 검색으로 각각 찾는다. 처리 순서는 다음과 같다.

1. 재작성된 사용자 질의를 입력으로 받는다.
2. Qdrant에서 Dense 검색으로 상위 50개를 가져온다.
3. OpenSearch에서 BM25 검색으로 상위 50개를 가져온다.
4. 두 검색 모두에 권한(acl) 필터를 적용한다.
5. 두 결과를 RRF(k=60)로 합쳐 후보 40개를 남긴다.
6. 리랭커(Cross-Encoder)가 후보 40개의 점수를 다시 계산한다.
7. 최종 상위 5개를 LLM에 전달한다.

RRF는 점수 대신 순위만 사용한다. 그래서 척도가 다른 두 검색 결과를 쉽게 합칠 수 있다 [3].

RRF 점수 공식은 RRF(d) = Σ 1 / (k + rank_i(d))이다. rank_i(d)는 i번째 검색 결과에서 문서 d의 순위다. k=60은 RRF 원 논문에서 사용한 값이다 [3].

7.6 Anthropic 공개 실험 결과 (참고)

아래 수치는 KnowBridge의 결과가 아니다. Anthropic이 공개한 실험 결과다 [1]. 지표는 top-20 검색 실패율이며, 낮을수록 좋다.

- 기존 임베딩: 5.7%
- Contextual Embedding 적용: 3.7% (35% 감소)
- Contextual Embedding + Contextual BM25: 2.9% (49% 감소)
- 위 구성 + 리랭킹: 1.9% (67% 감소)

이 실험의 리랭킹에는 Cohere 리랭커가 쓰였다. KnowBridge는 bge-reranker-v2-m3를 쓰므로 수치를 직접 비교할 수 없다.

8. LangGraph 에이전트 워크플로우

KnowBridge의 질의응답 흐름은 LangGraph의 StateGraph로 구현했다. 노드는 처리 단계이고, 엣지는 다음 단계로 가는 조건이다.

8.1 노드별 역할

- **route_query (Claude):** 질문이 문서 검색이 필요한지, 일반 대화인지 분류한다.
- **rewrite_query (Claude):** 대화 맥락과 약어를 풀어 검색용 질의로 바꾼다. 재시도 시에는 다른 표현을 쓴다.
- **retrieve (Qdrant, OpenSearch):** Dense와 BM25 검색을 실행하고 RRF로 합친다. 권한 필터를 적용한다.
- **rerank (Reranker):** 후보 40개를 Cross-Encoder로 다시 점수화해 상위 5개를 남긴다.
- **grade_documents (Claude):** 각 문서가 질문에 답하는 데 필요한지 판정한다.
- **generate (Claude):** 관련 문서만 근거로 답변을 쓰고 출처를 붙인다.
- **check_grounded (Claude):** 답변의 각 주장이 근거 문서에 있는지 검증한다.
- **direct_answer:** 검색이 필요 없는 일반 대화에 답한다.
- **fallback:** 근거가 부족함을 알리고 관련 문서 링크만 제공한다.

8.2 엣지(전이 조건)

원본 문서의 상태 그래프를 “출발 노드 → 도착 노드 (조건)” 형식으로 적었다.

1. START → route_query
2. route_query → rewrite_query (검색이 필요한 질문)
3. route_query → direct_answer (잡담·인사)
4. rewrite_query → retrieve
5. retrieve → rerank
6. rerank → grade_documents
7. grade_documents → generate (관련 문서가 2개 이상)
8. grade_documents → rewrite_query (관련 문서 부족, 재검색 2회 이내) — 재검색 루프
9. grade_documents → fallback (관련 문서 부족, 재검색 횟수 초과)
10. generate → check_grounded
11. check_grounded → END (근거 일치)
12. check_grounded → generate (근거 불일치, 재생성 1회 이내) — 재생성 루프
13. check_grounded → fallback (근거 불일치, 재생성 횟수 초과)
14. direct_answer → END
15. fallback → END

8.3 State 정의

KnowBridge 그래프의 모든 노드는 하나의 State를 읽고, 바뀐 필드만 반환한다.

```
from typing import TypedDict
from langgraph.graph import StateGraph, START, END

class RAGState(TypedDict):
    question: str          # 사용자의 원래 질문
    search_query: str      # rewrite_query가 만든 검색용 질의
    route: str             # "search" 또는 "chitchat"
    documents: list[dict]  # 리랭킹 후 문서와 관련성 평가 결과
    answer: str            # 생성된 답변
    grounded: bool         # 근거 검증 통과 여부
    search_count: int      # 검색 횟수, 첫 검색 포함 (최대 3)
    gen_count: int         # 생성 횟수, 첫 생성 포함 (최대 2)
```

8.4 그래프 구성 코드

```
def after_grade(state: RAGState) -> str:
    relevant = [d for d in state["documents"] if d["relevant"]]
    if len(relevant) >= 2:
        return "generate"
    return "retry" if state["search_count"] < 3 else "fallback"

def after_check(state: RAGState) -> str:
    if state["grounded"]:
        return "done"
    return "regenerate" if state["gen_count"] < 2 else "fallback"

g = StateGraph(RAGState)
for name, fn in [("route_query", route_query), ("rewrite_query", rewrite_query),
                 ("retrieve", retrieve), ("rerank", rerank),
                 ("grade_documents", grade_documents), ("generate", generate),
                 ("check_grounded", check_grounded),
                 ("direct_answer", direct_answer), ("fallback", fallback)]:
    g.add_node(name, fn)

g.add_edge(START, "route_query")
g.add_conditional_edges("route_query", lambda s: s["route"],
                        {"search": "rewrite_query", "chitchat": "direct_answer"})
g.add_edge("rewrite_query", "retrieve")
g.add_edge("retrieve", "rerank")
g.add_edge("rerank", "grade_documents")
g.add_conditional_edges("grade_documents", after_grade,
    {"generate": "generate", "retry": "rewrite_query", "fallback": "fallback"})
g.add_edge("generate", "check_grounded")
g.add_conditional_edges("check_grounded", after_check,
    {"done": END, "regenerate": "generate", "fallback": "fallback"})
g.add_edge("direct_answer", END)
g.add_edge("fallback", END)
app = g.compile()
```

search_count는 rewrite_query 노드가 실행될 때마다, gen_count는 generate 노드가 실행될 때마다 1씩 늘린다.

그래서 재검색은 최대 2회, 재생성은 최대 1회로 제한된다. 이 카운터가 무한 루프를 막는다.

9. 성능 평가

주의: 이 장의 평가 데이터와 수치는 모두 예시용 가상 값이다.

9.1 평가 데이터셋

KnowBridge 평가에는 질문 400개를 사용했다. 각 질문에는 정답 청크(golden chunk)를 사람이 지정했다.

질문 유형은 설정값 조회 140개, 장애 대응 절차 120개, 개념 설명 80개, 버전 비교 60개다.

9.2 평가 지표

검색 실패율@20은 정답 청크가 상위 20개 후보에 없는 질문의 비율이다. Anthropic 실험에서 쓴 지표와 같은 개념이다 [1].

답변 정확도는 평가자 2명이 독립 판정하고, 의견이 다르면 3번째 평가자가 결정했다.

9.3 구성별 결과

감소율은 (A의 실패율 − 해당 구성의 실패율) ÷ A의 실패율로 계산했다.

구성 A. 기본 임베딩 (프로토타입)

검색 실패율@20 15.0%, 답변 정확도 70.0%, 근거 충실도 0.80, P95 지연 1.8초.

구성 B. A + Contextual Embedding

검색 실패율@20 10.5%(30.0% 감소), 답변 정확도 77.5%, 근거 충실도 0.84, P95 지연 1.8초.

구성 C. B + Contextual BM25 (RRF 융합)

검색 실패율@20 8.0%(46.7% 감소), 답변 정확도 80.3%, 근거 충실도 0.86, P95 지연 2.0초.

구성 D. C + 리랭킹

검색 실패율@20 5.2%(65.3% 감소), 답변 정확도 85.0%, 근거 충실도 0.90, P95 지연 2.6초.

구성 E. D + LangGraph 재검색·검증 (최종)

검색 실패율@20 5.2%(65.3% 감소), 답변 정확도 88.5%, 근거 충실도 0.93, P95 지연 3.4초.

E는 검색 단계가 D와 같으므로 검색 실패율@20도 같다.

9.4 해석

가장 큰 개선은 Contextual Embedding 단계(A→B)에서 나왔다. 실패율이 4.5%p 줄었다. 문맥 유실이 주된 실패 원인이었기 때문이다.

Contextual BM25(B→C)는 실패율을 2.5%p, 리랭킹(C→D)은 2.8%p 더 줄였다.

LangGraph 재검색·검증(D→E)은 검색 실패율을 바꾸지 않았다. 대신 답변 정확도를 3.5%p 높이고 지연을 0.8초 늘렸다.

10. 운영 및 리스크 관리

10.1 증분 색인 절차

KnowBridge는 문서가 바뀌면 바뀐 문서만 다시 색인한다. 절차는 다음과 같다.

1. **변경 감지:** 웹훅과 일 1회 스캔으로 바뀐 문서를 찾는다.
2. **해시 비교:** content_hash를 비교해 실제로 내용이 바뀌었는지 확인한다.
3. **문서 전체 재처리:** 해당 문서를 다시 청킹하고 모든 청크를 다시 문맥화한다.
4. **교체 적재:** 기존 청크를 삭제한 뒤 새 청크를 적재한다.
5. **회귀 평가:** 샘플 50문항으로 검색 품질이 떨어지지 않았는지 확인한다.

바뀐 청크만이 아니라 문서의 모든 청크를 다시 문맥화한다. 문서 일부가 바뀌면 다른 청크의 문맥 설명도 달라질 수 있기 때문이다.

10.2 모니터링 지표

경보 기준값은 가상이다.

- **P95 응답 시간:** 사용자 체감 지연이다. 5분간 4초를 넘으면 경보를 보낸다.
- **fallback 비율:** “답변 불가” 응답의 비율이다. 급증하면 색인 누락을 의심한다. 일간 15%를 넘으면 경보를 보낸다.
- **재검색 루프 발생률:** 첫 검색에서 문서가 부족했던 비율이다. 일간 25%를 넘으면 경보를 보낸다.
- **사용자 부정 피드백:** 답변에 대한 싫어요 비율이다. 주간 10%를 넘으면 경보를 보낸다.
- **색인 지연:** 문서 변경부터 색인 완료까지 걸린 시간이다. 24시간을 넘으면 경보를 보낸다.

10.3 리스크와 대응

- **생성된 문맥의 사실 오류:** 잘못된 문맥이 엉뚱한 청크를 상위로 올린다. 대응으로 프롬프트에 “문서에 없는 사실 생성 금지”를 넣고, 매주 문맥 100건을 표본 검수한다.
- **문맥 생성 비용 증가:** 문서가 자주 바뀌면 재문맥화 비용이 커진다. 대응으로 프롬프트 캐싱을 쓰고, 변경이 잦은 문서는 일 1회로 묶어 처리한다.
- **권한 없는 문서 노출:** 보안 사고로 이어진다. 대응으로 검색 단계에서 acl 필터를 강제하고, 생성 단계에는 필터를 통과한 문서만 전달한다.
- **오래된 문서 검색:** 폐기된 버전의 절차를 안내한다. 대응으로 version과 updated_at 메타데이터로 최신 문서를 우선한다.
- **응답 지연 증가:** 재검색 루프가 지연을 늘린다. 대응으로 재검색은 최대 2회, 재생성은 최대 1회로 제한한다.

11. 결론, 향후 계획, 참고문헌

11.1 결론

KnowBridge는 LangGraph 기반 Agentic RAG로 사내 기술문서 질의응답 에이전트를 구현했다.

핵심 문제는 문서를 청크로 쪼갤 때 서비스명, 버전, 지시 대상 같은 문맥이 사라지는 것이었다.

Contextual Retriever로 청크마다 문맥을 붙인 뒤, 검색 실패율@20은 15.0%에서 5.2%로 줄었다(가상 수치).

LangGraph의 재검색 루프와 근거 검증은 답변 정확도를 88.5%까지 높였다(가상 수치).

11.2 향후 계획

- **표·이미지 문맥화:** 표와 다이어그램 이미지도 문맥 설명을 만들어 색인한다.
- **문맥 생성 모델 경량화:** 더 작은 모델로 문맥을 만들 때 검색 품질이 얼마나 떨어지는지 비교한다.
- **평가 자동화:** 문서 배포 때마다 회귀 평가를 자동 실행하고 결과를 대시보드에 올린다.
- **멀티 에이전트 확장:** 장애 대응 질문은 로그 조회 에이전트와 협업하도록 LangGraph 서브그래프를 추가한다.

11.3 참고문헌

1. Anthropic, “Introducing Contextual Retrieval,” 2024-09. https://www.anthropic.com/news/contextual-retrieval
2. LangGraph GitHub 저장소. https://github.com/langchain-ai/langgraph
3. G. V. Cormack, C. L. A. Clarke, S. Büttcher, “Reciprocal Rank Fusion outperforms Condorcet and individual Rank Learning Methods,” SIGIR 2009.
4. BAAI, bge-m3 모델 카드. https://huggingface.co/BAAI/bge-m3
5. Anthropic, Prompt caching 문서. https://docs.claude.com/en/docs/build-with-claude/prompt-caching

11.4 용어 정리

- **청크(chunk):** 검색을 위해 문서를 잘라 만든 작은 텍스트 조각이다.
- **문맥 유실:** 청크가 원문에서 떨어지며 주체, 버전, 지시 대상 정보를 잃는 현상이다.
- **Contextual Retrieval:** 청크마다 LLM이 만든 설명 문맥을 붙여 임베딩과 BM25 색인을 만드는 기법이다.
- **RRF:** 여러 검색 결과를 순위 기반 점수로 합치는 방법이다.